Architecture Transferability Quantifying for Multi-Task Neural Architecture Search

Tingjie Zhang, Hai-Lin Liu, *Senior Member, IEEE*, Songyi Xiao, and Lei Chen

***Abstract*—Multi-task neural architecture search (MTNAS) transfers architectural knowledge on neural models between different tasks, with the basic aim of optimizing neural architectures. MTNAS enables effective architectural patterns contained in the source task to be transferred and reused in the target task, which is known as transferability. However, ranking disorder between the source and target task degrades architecture transferability. To address this issue, we devise an architecture transferability quantifying method (ATQ). First, our data-agnostic method maps neural architectures into architecture embedding vectors for subsequent transferability prediction. Second, we apply transfer rank, an instance-based classifier, to alleviate transferability degradation problem. In this work, ATQ is incorporated into an evolutionary multi-task NAS algorithm (KTNAS), to select candidate architectures with better transferability. Extensive experiments are conducted on multiple benchmark datasets, such as NASBench-201, TransNAS-Bench-101 and DARTs. Experimental results show that KTNAS outperforms peer MTNAS algorithms in search efficiency and downstream task performance. Ablation study demonstrates the vital importance of transfer rank on enhancing transfer performance.**

***Index Terms*—Multi-task neural architecture search, evolutionary multi-tasking optimization, architecture transferability quantifying.**

I. Introduction

NEURAL architecture search (NAS) has achieved remarkable success in designing state-of-the-art networks for tasks like image classification [1], [2], [3], [4], object detection [5], and semantic segmentation [6]. However, NAS works well only for a specific task. When the task changes, NAS needs to learn from scratch, which is expensive in real-world applications.

In such cases, knowledge transfer is applied among multiple tasks to reduce unnecessary search costs. This process involves extracting knowledge from source tasks and applying it to a target task, which effectively enhances performance, primarily through data augmentation or model/feature reuse [7]. For instance, in the common fine-tuning approach illustrated in Fig. 1(a), the entire network architecture and its pretrained weights from the source task are reused for the target task, typically followed by several steps of additional training to adapt the model.

The architecture searched by NAS algorithms over different tasks exhibits similarity and transferability. For example, the backbone designed for CIFAR-10 can be easily reused by other related tasks, such as CIFAR-100 and ImageNet [8]. In other words, architectural similarity enables the architectural knowledge extracted from the source task to be preserved, reused and refined to similar tasks [9]. The parts of cells that do matter for architecture performance often follow similar and simple patterns, which make them effective in transfer scenarios [10]. Our work seeks to identify architectures most likely to possess the common patterns, maximizing the probability of positive transfer.

Meanwhile, task differences also weaken architecture transferability, which justifies the necessity of measuring transferability. In cross-task scenarios, studies have found that different downstream tasks, such as classification, segmentation, keypoint detection, tend to have different optimal architectures. Directly using an architecture searched for image classification may achieve suboptimal results on other tasks [11]. Architectures that perform similarly on the source dataset may show significant differences in performance when transferred to a large target dataset (such as ImageNet) [12]. Intuitively, ranking architectures by their performance on the target dataset can effectively improve transfer success rate and final performance [13].

Beyond task-specific self-evolution, multi-task NAS facilitates cross-task knowledge transfer by sharing useful architectural components across different search processes. As shown in Fig. 1(b), EMT-NAS [14] utilizes the knowledge-sharing method proposed in MFEA [15] for performing crossover operation between architectures belonging to different tasks, to accelerate multiple separated search processes. Besides, the algorithm maintains a personalized architecture and corresponding weights for each task to alleviate catastrophic forgetting [16]. Another parallel work MT-NAS [14] devises an adaptive transfer frequency to trade off the self-evolution and knowledge transfer for alleviating the potential negative transfer. In addition, a low-fidelity evaluation strategy is used to accelerate knowledge extraction.

Existing MTNAS algorithms, such as EMT-NAS, MT-NAS, adopt implicit architecture transfer (IAT), where transferable architectures are randomly sampled from promising

Received 1 May 2025; revised 11 November 2025; accepted 11 December 2025. Date of publication 20 January 2026; date of current version 26 March 2026. This work was supported by the National Natural Science Foundation of China under Grant 62172110. *(Corresponding author: Hai-Lin Liu.)*

Tingjie Zhang, Hai-Lin Liu, and Lei Chen are with the School of Mathematics and Statistics, Guangdong University of Technology, Guangzhou 510006, China (e-mail: ztj@mail2.gdut.edu.cn; hlliu@gdut.edu.cn; Chenlei3@gdut.edu.cn).

Songyi Xiao is with the School of Automation, Guangdong University of Technology, Guangzhou 510006, China (e-mail: 1112104020@mail2.gdut.edu.cn).

Recommended for acceptance by J. Ding.

Digital Object Identifier 10.1109/TETCI.2025.3647652

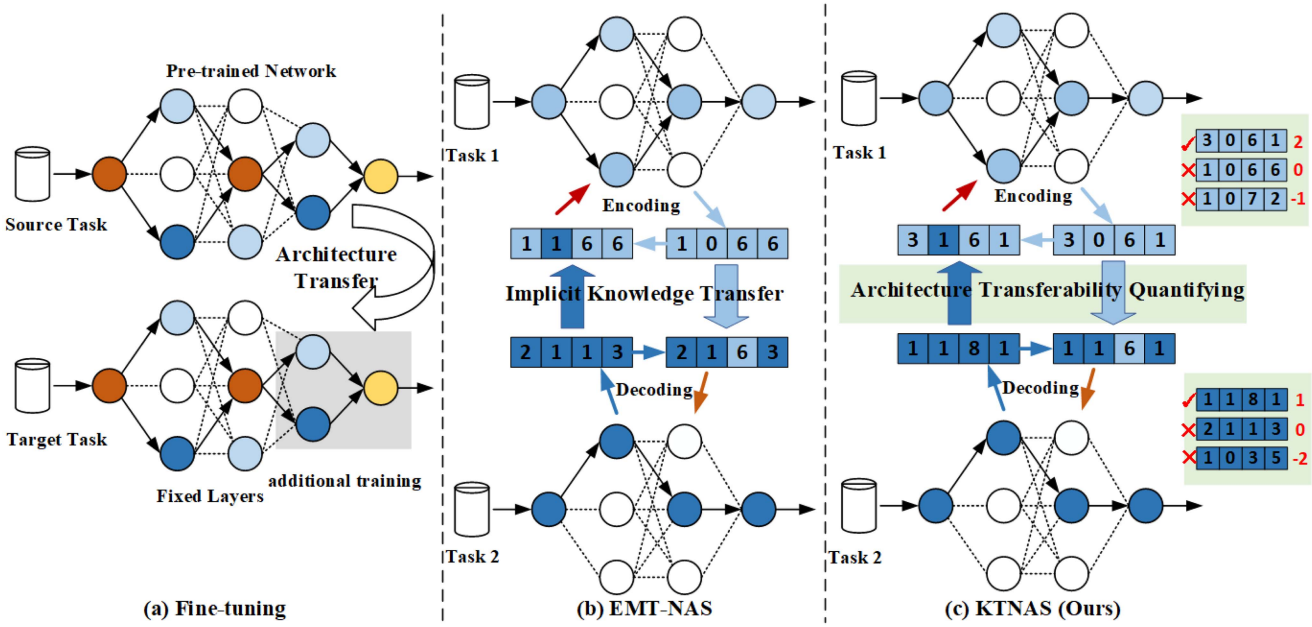

Fig. 1. A comparison of different knowledge transfer processes. (a) Fine-tuning reuses the entire architecture and partial weights. (b) EMT-NAS selects transferable architectures through random sampling. (c) KTNAS (Ours) selects transferable architectures with a high transfer rank. Best viewed in color.

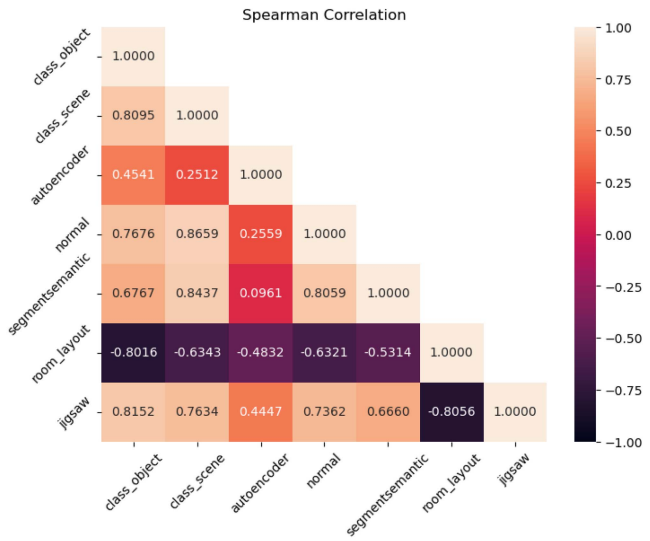

Fig. 2. Ranking correlation of transferred architectures among 7 tasks on Micro TransNAS-Bench-101.

architectures of source task. However, the consistency of architecture rankings between source and target tasks, as measured by the Spearman correlation coefficient (θ), is not always high. Of 21 tasks of TransNAS-Bench-101 in Fig. 2, only 10 have high correlation ($\theta > 0.7$), 3 have low correlation ($\theta < 0.3$), and 8 have moderate correlation ($0.3 \leq \theta \leq 0.7$). Small ranking correlation may weaken the transfer effect on downstream tasks, namely transferred architectures perform well on the source task while poorly on the target task. In such case, not helpful transferred architectures also lead to additional computation costs or even disrupts the learning of target task. IAT will bring some negative influence on the learning of target task.

To address this critical issue, we propose architecture transferability quantifying (ATQ), a novel knowledge transfer module designed to replace the unreliable random sampling of IAT. ATQ performs an explicit quantification of transfer potential for an architecture. Inspired by MMOTK [17] from evolutionary multi-task optimization, our approach consists of two key stages. First, we employ the arch2vec [18] algorithm to map each neural architecture into a continuous embedding vector, which provides an analysis basis for transferability quantification. Second, we calculate transferability ranking via transfer rank—a metric designed to rank architectures based on their predicted performance on the target task, using these embeddings.

In our proposed framework, ATQ acts as an intelligent filter to guide the knowledge sharing process. We first use transferability ranking to identify and select only those source architectures with high transferability, effectively eliminating candidates likely to cause negative transfer. Then, these high-potential architectures are used in a crossover operation with architectures from the target task. The target task can genuinely reuse the actually useful architectural components to accelerate the self-evolutionary search process and ultimately improve the performance learning of target task.

We summarize our contributions as follows:

- We propose ATQ with the joint use of arch2vec and transfer rank, which is incorporated into MTNAS algorithm (KTNAS) to achieve effective knowledge transfer and mitigate negative transfer.
- For explicit quantification of architecture transferability, we tailor transfer rank to suit architecture space with architectural similarity metric and positive transfer concept.
- Extensive experiments show that KTNAS outperforms existing multi-task methods. We further validate our approach

through detailed ablation studies and transfer performance analysis.

We review background and related work in Section II and present the details of proposed KTNAS in Section III. Experiment settings and results analysis are provided in Section IV.

II. Background and Related Work

A. MTNAS

Single-task NAS (STNAS) searches for an optimal architecture for a specific task. This process can be formulated as a bilevel optimization problem [19], where the architecture and its corresponding weights are optimized on the validation and training datasets, respectively:

$$\begin{aligned} &\min_{\alpha} L_(\omega^*(\alpha), \alpha; D_{val}) \\ &\text{s.t. } \omega^*(\alpha) = \arg\min_{\omega} L_(\omega, \alpha; D_{trn}) \end{aligned} \tag{1}$$

where the upper-level variable α denotes the candidate architecture, the lower-level variable ω denotes the corresponding weights, L is the task-specific loss function, and D_{trn}, D_{val} are the training and validation datasets.

Based on their search strategies, STNAS methods are typically categorized into three classes: reinforcement learning-based (RLNAS) [8], [20], gradient-based (GONAS) [19], [21], and evolution-based (EvoNAS) [22], [23], [24], [25], [26], [27], [28], [29]. We define transferability that effective architectural patterns from a source task can be transferred and reused for a target task. Compared to RLNAS and GONAS, EvoNAS is particularly well-suited for transferring architectural knowledge via crossover operations.

Multi-task NAS (MTNAS) extends STNAS to optimize multiple tasks simultaneously. In this paradigm, knowledge transfer occurs during the joint optimization of architectures (the upper-level optimization in (1)). For instance, MTNAS searchs for three tasks (image classification, scene segmentation, autoencoding) at once and performs knowledge transfer even with different loss functions (cross-entropy loss for classification, mIoU for segmentation, SSIM for autoencoding).

For concise expression, the training of weights on each task (the lower optimization in (1), corresponding to D_{trn}) is omitted in the following formulation. Given N tasks, a validation dataset D^i_{val} for the i-th task, MTNAS can be formulated as an EMTO problem:

$$\begin{cases} \alpha_1^* = argmin_{\alpha_1} L(\omega_1(\alpha_1), \alpha_1; D^1_{val}) \\ \alpha_2^* = argmin_{\alpha_2} L(\omega_2(\alpha_2), \alpha_2; D^2_{val}) \\ \quad \cdots \\ \alpha_N^* = argmin_{\alpha_N} L(\omega_N(\alpha_N), \alpha_N; D^N_{val}) \\ \quad s.t.\ \alpha_i \in \Omega, i = 1, 2, \ldots, N \end{cases} \tag{2}$$

Ω denotes a unified search space that facilitates architectural knowledge transfer and reuse.

Knowledge transfer in MTNAS can be divided into two categories. The first class is unidirectional transfer, where external knowledge is used to benefit a target task. For example, a cell structure learned on CIFAR-10 can be transferred to other image classification tasks like CIFAR-100 and ImageNet [8]. Similarly, fine-tuning, as shown in Fig. 1(a), is a simple form of model-based transfer that enables model/feature reuse.

The second class is bidirectional transfer, with internal knowledge to improve the performance of different tasks. For instance, [30] searches for a more generalized cell by averaging controller rewards across tasks. MT-ENAS [31] proposes a surrogate model and a cross-task interaction layer to combine knowledge from multiple tasks. CSTA-ENAS [32] focuses on finding transferable models for edge devices using a cross-task performance metric. EMT-NAS [14] select architectures for transfer based on their validation accuracy on the source task, which is defective due to the ranking disorder problem mentioned previously. Another parallel work MT-NAS [14] uses an adaptive transfer frequency and low-fidelity evaluation to enhance search efficiency.

As a concurrent work, our proposed KTNAS addresses a key limitation of MTNAS in existing knowledge-transfer strategies. As illustrated in Fig. 1(c), KTNAS enables effective knowledge transfer by ATQ, a more accurate individual selection mechanism. Experimental results demonstrate that KTNAS can effectively alleviate negative transfer and improve performance on the target tasks.

B. EMTO

Evolutionary Multi-task Optimization (EMTO) utilizes evolutionary algorithms (EAs) to facilitate knowledge transfer across different tasks during their simultaneous optimization. Due to their powerful global optimization characteristics, EAs are widely used in many fields, such as defect detection [33], many-objective optimization [34], transportation network optimization [35], etc. The common practice of EMTO is to build an independent population for each task [15]. Each population has two behaviors: task-specific self-evolution and cross-task knowledge transfer. For the latter, various knowledge-sharing mechanisms have been investigated [17], [36], [37], [38], which involve identifying, refining, or directly injecting promising solutions from source tasks into a target population.

When pair tasks share similarities, promising solutions from one can be beneficial to another. Various methods have been proposed to leverage this characteristic. For example, MO-MFEA [36] transforms solutions into a unified space to facilitate reuse. EMEA [37] directly transfers non-dominated solutions between highly similar tasks. EMTIL [38] employs a Bayesian classifier with incremental learning to classify candidate into categories: positive-transfer and negative-transfer solutions. MMOTK [17] introduces transfer rank, a supervised instance-based model designed to quantify the transfer priority of each solution. Consequently, only Solutions with a high transfer rank are selected for knowledge transfer, enabling a more accurate selection process.

In this work, we integrate the concept of transfer rank to guide the knowledge extraction process. This allows us to move beyond heuristic/random selection and instead implement a more precise and effective mechanism for identifying high-potential solutions for cross-task knowledge sharing.

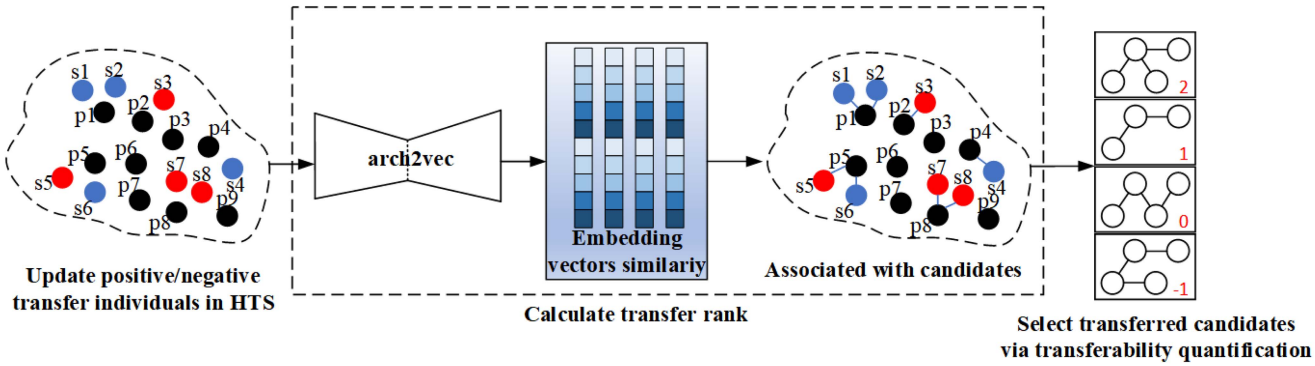

Fig. 3. Details of architecture transferability quantifying method.

C. Architecture Embedding

A neural architecture can be typically represented as a directed acyclic graph (DAG), where nodes represent computational operations and edges denote the flow of information between nodes. The goal of architecture embedding is to map these complex structure characteristics into a low-dimensional latent space. In this space, architectures with similar performance are positioned closely together, forming meaningful clusters. From a feature engineering perspective, this process effectively reduces the dimensionality of architectural representations, yielding resulting vectors that can be utilized in downstream tasks.

Several embedding methods are investigated. node2vec [39], a general-purpose graph embedding technique, learns node representations by exploring their diverse network neighborhoods. Tailed to neural architecture, arch2vec [18] utilizes variational graph isomorphism autoencoder to map network architectures into a latent space that is more strongly correlated with architecture performance. CATE [40] uses a transformer-based model with cross-attention to generate architecture encodings.

We select arch2vec as our architecture embedding method, primarily for its proven efficiency and unsupervised learning capability. This choice is empirically justified in Section IV-G, where an ablation study demonstrates its superior performance compared to alternative methods like node2vec and CATE.

III. KTNAS

In this section, we first provide an overview of the KTNAS framework and the ATQ module. The next is a discussion of two key components: the architecture similarity representation and the concept of positive transfer. We finally give the definition, update and selection of transfer rank.

A. Framework

The framework of KTNAS is presented in Algorithm 1. The algorithm contains several hyperparameters defined in previous sections: N, D_{trn}, D_{val}, K in Section II-A; $r\%$ in Section III-D; and m, n in Section III-E. T denotes the maximum number of generations. For each task, the initialization, training and evaluation of the current population P and the transfer population TP are performed at the zeroth generation (line 1-5). The steps of line 7-21 are repeated $T-1$ times and finally the best-performing individual for each task is output (line 19). First, the algorithm calculates transfer rank is used to select candidate individuals from other tasks (line 8-9). Then, crossover, mutation, training, and evaluation are performed sequentially, followed by the environmental selection mechanism from [14] (line 10–15). At last, the individual with the highest fitness in each task is recorded as a candidate for optimal architecture (line 16-17).

B. Architecture Transferability Quantifying

As depicted in Fig. 3 and detailed in Algorithm 2, the ATQ mechanism ranks the transferability of candidate architectures by mining common patterns between them and the historically transferred architectures. First, ATQ maintains a historical transferred set HTS for each task, which stores transfer information from the previous generations. Second, candidate and previously transferred architectures are converted into embedding vectors via arch2vec and transferability ranking is generated by calculating the transfer rank between these embeddings. Two key components tailored for neural architectures, a similarity metric sim, and a positive transfer criterion $label$, are discussed below.

C. Architecture Embeddings Similarity

The discrete encoding poses a challenge of quantifying transferability. To address this, we employ a variational graph isomorphism autoencoder, as proposed in arch2vec [18], to map each architecture into a continuous vector space. Compared to discrete encoding, this unsupervised approach ensures high scalability in clustering architectures that share similar transferable structures. Furthermore, arch2vec is trained solely on the architectures themselves, without requiring performance metrics like accuracy as labels.

We quantify the structure (i.e., topology and components) similarity between any pair of architectures using their corresponding vector representations. To this end, we define the architectural similarity metric sim based on the cosine similarity between their embeddings:

$$sim(\alpha_1, \alpha_2) = \cos(arch2vec(\alpha_1), arch2vec(\alpha_2)) \quad (3)$$

Algorithm 1: The Framework of KTNAS.

Input: Search space $\mathcal{A}$; Number of tasks N; Population size K; Transfer size n; Number of generations T; Datasets for each task $\{D^i_{train}, D^i_{val}\}_{i=1}^N$
Output: The set of best-found architectures, one for each task $\{p^1_{best}, \ldots, p^N_{best}\}$

for $i \leftarrow 1$ to N **do**
  $P^i_0 \leftarrow$ Initialize population with K random architectures from $\mathcal{A}$
  Train and evaluate all individuals in P^i_0 on D^i_{train} and D^i_{val} to get their fitness
  $p^i_{best} \leftarrow$ The individual with the highest fitness in P^i_0
end for
for $t \leftarrow 0$ to $T-1$ **do**
  for $i \leftarrow 1$ to N **do**
    // Generate transfer population using other tasks' parents
    $TP^i_t \leftarrow$ Generate transfer population of size n from $\{P^j_t\}_{j \neq i}$ using Algorithm 2
    // Generate offspring
    $O^i_t \leftarrow$ Apply crossover and mutation to individuals in $P^i_t \cup TP^i_t$
    // Evaluate new individuals
    Train and evaluate all new individuals in $O^i_t \cup TP^i_t$ on D^i_{train} and D^i_{val} to get their fitness
    // Environmental selection for the next generation
    $P^i_{t+1} \leftarrow$ Select the top-K individuals from $P^i_t \cup O^i_t \cup TP^i_t$ based on fitness
    // Update the best-found architecture for the task
    Let $(p^i_{best})_t$ be the individual with the highest fitness in P^i_{t+1}
    if $fitness((p^i_{best})_t) > fitness(p^i_{best})$ **then**
      $p^i_{best} \leftarrow (p^i_{best})_t$
    end if
  end for
end for

Algorithm 2: Quantifying Architecture Transferability.

Input: Target size of transfer population n, Number of historical generations to save m, Architectural similarity metric $sim(\cdot,\cdot)$, A function $label(s)$ that returns a label $\{+1, -1\}$ indicating positive/negative transfer, Current generation index t, Previous transfer population TP^{t-1}, Current population $P = \{p_1, \ldots, p_K\}$ of size K, Historical sets of negative transfers $\{Neg_1, \ldots, Neg_{t-1}\}$.
Output: New transfer population TP^t.

// Update the Historical Transfer Set
for each individual $s \in TP^{t-1}$ **do**
  if $label(s) = +1$ **then**
    $Pos \leftarrow Pos \cup \{s\}$
  else
    $Neg_t \leftarrow Neg_t \cup \{s\}$
  end if
end for
if $t \leq m$ **then**
  $HTS \leftarrow Pos \cup (\bigcup_{h=1}^{t} Neg_h)$
else
  $HTS \leftarrow Pos \cup (\bigcup_{h=t-m+1}^{t} Neg_{t-m+h})$
end if
// Calculate transfer rank for current population P
Let $L = |HTS|$.
Calculate the similarity matrix $D_{L\times K}$ where $D_{ij} = sim(s_i, p_j)$ for $s_i \in HTS, p_j \in P$.
Initialize associated sets $\Phi_j \leftarrow \emptyset$ and transfer ranks $\phi_j \leftarrow 0$ for all $p_j \in P$.
for $i \leftarrow 1$ to N **do**
  // Find the most similar individual p_j for s_i
  $k \leftarrow \arg\max_{j\in\{1,\ldots,K\}} D_{ij}$
  $\Phi_k \leftarrow \Phi_k \cup \{s_i\}$
end for
for $j \leftarrow 1$ to K **do**
  $\phi_j \leftarrow \sum_{s\in\Phi_j} label(s)$
end for
// Select individuals for the new transfer population
Group individuals in P into disjoint sets $A_1, \ldots, A_H$ based on their rank ϕ, sorted in descending order.
Find the largest index h such that $|\bigcup_{i=1}^{h} A_i| \leq n$.
$TP^t \leftarrow \bigcup_{i=1}^{h} A_i$.
if $|TP^t| < n$ and $h < H$ **then**
  Randomly select $n - |TP^t|$ individuals from the next rank group A_{h+1} and add them to TP^t.
end if

where α_1 and α_2 denote pair architectures, and cos represents the cosine similarity function. The value of sim ranges from 0 to 1; a larger value indicates higher similarity, whereas a smaller value indicates lower similarity.

D. Concept of Positive Transfer

The concepts of positive and negative transfer are defined within the context of architecture transfer. A positive transfer occurs when a transferred architecture contributes beneficial genetic material—through crossover operation —that enhances the performance of the offspring population in the target task. A negative transfer is the opposite.

Existing positive transfer methods either have a high cost of evaluating architecture performance [37], or are not precise enough in dividing the transferability of candidate architectures [38]. To circumvent these costly evaluations, we propose an indirect yet effective method to assess transferability. Instead of evaluating the transferred architecture itself, we measure its value by the performance of its offspring. This approach is predicated on the strong performance correlation observed between parent and child architectures, where beneficial genetic material (e.g., effective topologies or components) is inherited through reproduction. We classify a transferred parent architecture as positive/negative based on the children's ranking in the target task's evolving population. Consequently, positive transfer assessment accelerates the discovery of high-performing architectures with almost no additional cost.

We note that a parent population P with population size K, a transfer population TP extracted from other tasks. Offspring population O is generated by applying reproduction operators on individuals of $P \cup TP$. We introduce a hyperparameter, elite ranking ratio $r\%$, as the ranking threshold for positive transfer assessment. A transferred parent architecture $s \in TP$ is classified as positive if at least one of its children ranks within the top $r\%$ of the next-generation population Z. Otherwise, s is classified as negative. This is defined as:

$$\text{label}(s) = \begin{cases} 1 & \text{if } \exists o \in children(s)\ s.t.\ rank(o, Z) \leq r\% \times |Z|, \\ -1 & \text{otherwise.} \end{cases} \tag{4}$$

where $children(s)$ is the set of offspring generated from parent s, and $rank(o, Z)$ is the performance rank of individual o within population Z.

E. Transfer Rank Calculation

This section introduces the transfer rank, a metric designed to quantify architectural transferability. Our proposed ATQ method leverages this rank to identify and select promising architectures from source tasks for transfer to a target task. The core idea is to predict the transfer potential of a candidate by associating the historical performance of its structurally similar, previously transferred neighbors. The calculation of transfer rank in Algorithm 2 comprises three steps:

1. Maintenance of historical transferred set (HTS): For each task, a Historical Transfer Set (HTS) is maintained (lines 1-13), which stores the architectural encodings and corresponding binary transfer labels (+1 for positive, -1 for negative) of all previously transferred individuals. To keep the class balance, negative-transfer individuals are retained using a sliding window of size m, where m is a hyperparameter defining the number of recent generations to consider. A larger value increases the pool of negative samples available for a more accurate classification.

2. Transfer rank calculation: The transfer rank serves as a training-free, low-fidelity surrogate model (lines 14-25), ensuring high efficiency. To compute the rank for a candidate architecture, we first find its nearest neighbors in the historical transfer set via a similarity metric sim (lines 18-22). The candidate's rank is then obtained by the accumulated transfer labels of these neighbors.

3. Selection of transferred architectures: New transferred architectures are selected based on their transfer ranks to form the transfer population for the next generation (line 26-32).

An example of this calculation is illustrated in Fig. 4. Consider a candidate population $P = \{p_1, \ldots, p_9\}$ and a transfer population from a source task $TP = \{s_1, \ldots, s_8\}$.

Step 1. Labeling: Classify the previously transferred individuals. s_3, s_5, s_7, s_8 are positive, s_1, s_2, s_4, s_6 are negative.

Step 2. Association: Each $s \in TP$ is associated with its nearest neighbor in P. s_1, s_2 associate with p_1; s_3 associates with p_2, s_4 associates with p_4; s_5, s_6 associate with p_5, s_7, s_8 associate with p_8.

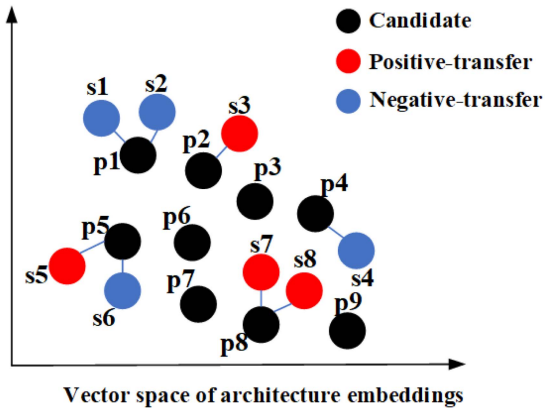

Fig. 4. An example of transfer rank.

Step 3. Rank accumulation: Each $p \in P$ accumulates the labels from its associated architectures in TP. For instance, p_1 is associated with $s_1(-1)$ and $s_2(-1)$, $-1-1 = -2$; p_2 is associated with $s_3(1)$, 1; p_4 is associated with $s_4(-1)$, -1; p_5 is associated with $s_5(+1)$ and $s_6(-1)$, $1-1 = 0$; p_8 is associated with $s_7(+1)$, $s_8(+1)$, $1+1 = 2$; p_8 is associated with $s_7(+1)$ and $s_8(+1)$, $1+1 = 2$; p_3, p_6, p_7, p_9 have no associations, 0.

Step 4. Selection: Transferability ranking: $p_8(2) > p_2(1) > p_3, p_5, p_6, p_7, p_9(0) > p4(-1) > p_1(-2)$. If we need to select $n = 2$ architectures, we choose p_1 and p_4. If $n = 3$, the third architecture will be chosen randomly from the next-highest rank group $(p_3, p_5, p_6, p_7, p_9)$. These selected architectures then form the new transfer population. Notebaly, if all candidates in the transfer pool are classified as non-positive ($\phi \leq 0$), the transfer process is skipped in that generation, preventing harmful genetic material from entering the population.

IV. Experimental Results

A. Search Spaces and Hyperparameters

NASBench-201 (NAS201) [12]: NAS201 is a cell-based NAS benchmark, containing 15,625 candidate architectures, where each cell is constructed from 4 nodes and 5 distinct operation choices. NAS201 provides evaluated performance metrics for every architecture on three datasets: CIFAR-10, CIFAR-100, and ImageNet-16-120.

TransNAS-Bench-101 (Trans101) [11]: Trans101 is a cell-based benchmark designed for cross-task NAS, with 4,096 unique architectures (referred to as backbones) and provides their performance across seven distinct computer vision tasks. This allows for the evaluation of cross-task search efficiency and the transferability of the discovered architectures.

DARTs [19]: Following EMT-NAS [14], we utilize a unified, DARTs-based search space designed for knowledge transfer across tasks. A cell is composed of 5 blocks and 9 candidate operations. During the search and evaluation stages, beyond adopting the same architecture encoding and hyperparameter settings as the baseline, our method introduces a distinct knowledge-extraction strategy namely ATQ. We conduct experiments on three dataset pairs: CIFAR-10/100,

TABLE I
HYPER-PARAMETERS SETTING ON 3 SEARCH SPACES

Parameters	NAS201	Trans101	DARTs
EvoNAS method	REA [24]	GCN [41]	GA
Low-fidelity evaluation	zero-cost [42]	GCN-based	weight sharing
Population size	10	10	20
Tournament size	5	5	2
Crossover probability	-	-	1
Mutation probability	1	1	0.02
Elite ranking ratio	20%	20%	20%
# Saved generations	5	5	5
# Transferred individuals	4	4	4

TABLE II
COMPARISON ON THE NUMBER OF EVALUATIONS FOR FINDING THE BEST ARCHITECTURES ON NAS201

Method	C-10	C-100	ImageNet	Total
RS	7782.1	7621.2	7726.1	23129.4
REA [24]	563.2	438.2	715.1	1439.6
GCN [41]	214.4	260.5	250.6	725.5
LaNAS [43]	247.1	187.5	292.4	727.0
WeakNAS [44]	182.1	78.4	268.4	528.9
Zero [42]	230.8	18.4	213.0	462.2
MT-NAS [9]	106.0	30.1	92.3	228.4
KTNAS (Ours)	**100.7**	**27.9**	**60.0**	**188.5**

MNIST/Fashion-MNIST, and MedMNIST, which comprises four sub-datasets (PathMNIST and OrganMNIST_Axial, Coronal, Sagittal). These are abbreviated as C-10/100, MNIST/F-MNIST, and Path/Organ_A,C,S, respectively.

Embedding dimension: Follow the pretraining settings of [18], we extract the 128-dimensional embedding vectors from the encoder's output. These embeddings are then used for quantifying transferability.

Metrics: We evaluate search efficiency using two metrics. For the NAS201 and Trans101 benchmarks, we report the number of architecture evaluations conducted during the search stage. For the DARTs-based search space, we report the total search time in GPU Days.

Experimental setup: On NAS201 and Trans101, we perform KTNAS 10 times with different random seeds. For the DARTs-based experiments, we report the mean and standard deviation over 5 independent runs; in this setting, all discovered architectures are trained from scratch for final evaluation. All experiments were conducted on a single NVIDIA RTX 3090 GPU with 24 GB of VRAM. Besides, the hyperparameters of evolution and transfer processes are listed in Table I.

B. NASBench-201 Result

Assessing search efficiency, we compare our proposed KTNAS with popular STNAS algorithms like Random Search (RS), Regularized Evolution (REA) [24], and GCN [41], as well as low-fidelity algorithms like LaNAS [43], WeakNAS [44], and Zero [42]. MTNAS [9] is also included as baseline multi-task NAS algorithm.

On 10 independent runs, we report the average number of evaluations required to find the optimal architecture in Table II. KTNAS is significantly more efficient than other compared methods. It requires 2-120× fewer total evaluations than STNAS and low-fidelity algorithms. Specifically, KTNAS achieves a remarkable 120× reduction in search cost compared to RS. Compared with MT-NAS, KTNAS consistently demonstrates lower search costs on each individual task and total evaluations.

TABLE III
COMPARISON ON ARCHITECTURE PERFORMANCE WHEN LIMITING THE NUMBER OF EVALUATIONS ON TRANS101

Search Method	Object C. Acc (%) ↑	Scene C. Acc (%) ↑	Room Lay. L2 Loss↓	Average Rank↓
RS	45.16	54.41	61.48	57.7
REA [24]	45.39	54.62	61.75	44.0
NSGA-NetV2 [45]	45.61	54.75	61.73	36.3
WeakNAS [44]	45.60	54.72	60.31	15.0
MT-NAS [9]	46.05	54.85	60.07	5.7
KTNAS (Ours)	**46.07**	**54.90**	**60.10**	**5.5**
Optimal	46.32	54.94	59.38	1.0

C. TransNAS-Bench-101 Result

To assess the cross-task generalization of discovered architectures, we limit each search process to 50 evaluations. We search for a backbone network and evaluate it on three distinct vision tasks from the TransNAS-Bench-101 suite: object classification, scene classification, and room layout estimation. As shown in Table III, we report the average performance over 10 independent runs, comparing our proposed KTNAS with a representative set of baselines. These include STNAS algorithms (REA, NSGA-NetV2), a low-fidelity algorithm (WeakNAS), and a multi-task algorithms (MT-NAS).

The results demonstrate that KTNAS achieves state-of-the-art performance, with the best/lowest average rank of 5.5 across the three tasks. Compared with MT-NAS, KTNAS can achieve comparable evaluation results on room layout (60.10% vs. 60.07%), while beats it on objective classification (46.07% vs. 46.05%) and scene classification (54.90% vs. 54.85%), confirming its effectiveness even with a limited search budget.

D. DARTs Results

To demonstrate the scalability of our approach, we evaluate KTNAS not only on two popular training-free benchmarks (NAS-Bench-201 and TransNAS-Bench-101) but also within the real-world DARTS search space.

CIFAR-10/100 result: For this experiment, multi-task algorithms search simultaneously on CIFAR-10 and CIFAR-100, where EMTO methods facilitate cross-task knowledge transfer. The comprehensive comparison results are presented in Table IV.

The EMTO-based algorithms (EMT-NAS, MT-NAS, and KTNAS) generally achieve higher classification accuracy than most single-task methods, demonstrating the effectiveness of architectural knowledge transfer. While some specialized single-task methods like NASNet-A, ReCNAS, achieve slightly higher accuracy on CIFAR-10, KTNAS offers a superior balance of performance and efficiency. For instance, compared to NASNet-A, KTNAS achieves a competitive accuracy of 97.12% with a

TABLE IV
COMPARISON RESULTS ON C-10/100. * INDICATES THAT FOUND ARCHITECTURE ON C-10 IS TRANSFERRED TO C-100, SO GPU DAYS SEARCHED ON C-10 AND C-100 ARE THE SAME

Architecture	Search Method	GPU Days	C-10		C-100	
			Params (M)	Test Acc (%)	Params (M)	Test Acc (%)
Wide ResNet [2]	manual	-	36.5	95.83	*	79.50
DenseNet [3]	manual	-	27.2	96.26	*	80.75
MobileNetV2 [4]	manual	-	2.1	94.56	*	77.09
PNAS [46]	SMBO	150	3.2	96.59±0.09	*	80.47
NASNet-A [8]	RL	2000	3.3	97.35	-	-
ENAS [20]	RL	0.45	4.6	97.11	*	80.57
DARTS [19]	GO	1.5	3.3	97.00±0.14	*	79.48±0.31
SNAS [21]	GO	1.5	2.9	97.02	-	-
ReCNAS [47]	GO	0.3	4.1	97.48±0.09	-	-
large-scale Evo [22]	EA	2750	5.4	94.6	40.4	77
Hier-EA [23]	EA	300	64	96.25±0.12	-	-
Amoebanet-A [24]	EA	3150	3.2	96.66±0.06	*	81.07
NSGA-Net [27]	EA	8	3.3	96.15	*	79.26
CARS-E [28]	EA	0.4	3.0	97.14	-	-
SMCSO [48]	EA	1.32	3.46	97.12	3.72	80.66
EMT-NAS [14]	EMTO	**0.42**	2.91	97.04±0.04	2.97	82.60±0.38
MT-NAS [9]	EMTO	0.50	-	**97.25±0.04**	-	82.55±0.45
KTNAS (Ours)	EMTO	0.48	2.97	97.12±0.10	2.92	**82.93±0.19**

TABLE V
COMPARISON RESULTS ON MNIST/F-MNIST

# Tasks	Model	Search Method	GPU Days (%)↓	MNIST (%)	F-MNIST (%)
1	CTM [49]	manual	-	99.4	91.4
	FastSNN [50]	manual	-	99.30	90.57
	NeuPDE [51]	manual	-	99.49	92.4
	ResNet-18 [1]	manual	-	99.69	93.35
	WaveMix-128/7 [52]	manual	-	99.71	93.91
2	Single-Tasking (Baseline) [14]	EA	+00.00	99.40±0.13	93.70±0.42
	EMT-NAS [14]	EMTO	+2.65	99.40±0.09	93.86±0.68
	KTNAS (Ours)	EMTO	**-10.28**	**99.62±0.17**	**94.36±0.46**

remarkable 4166× reduction in search cost. KTNAS reaches comparative classification accuracy and search time, yet with a model size merely two-thirds that of ENAS.

Compared with multi-task counterparts, KTNAS outperforms EMT-NAS by 0.33% and MT-NAS by 0.38% in terms of C-100 test accuracy with slightly small model size. We perform ablation study (KTNAS using random architecture selection). Compared with KTNAS, we observe that transfer rank help to find a better model for performance-complexity trade-off.

MNIST/F-MNIST result: The comparison results on MNIST and F-MNIST are presented in Table V. In this table, the search cost, reported as "GPU Days (%)", is normalized relative to the Single-Tasking baseline. When compared with manually designed models, the EMTO-based methods demonstrate competitive performance on MNIST and superior performance on F-MNIST. We speculate that this is because manually designed architectures have already reached a point of performance saturation on the relatively simple MNIST dataset, whereas multi-task NAS can more effectively explore the search space to find better solutions for the more challenging F-MNIST task.

Focusing on the multi-task comparison, KTNAS significantly outperforms EMT-NAS, achieving a 0.22% higher accuracy on MNIST and a 0.50% improvement on F-MNIST. Notably, these accuracy gains are achieved with significantly greater efficiency. KTNAS reduces the search cost by 10.28% relative to the single-tasking baseline, whereas EMT-NAS increases it by 2.65%, resulting in a total efficiency improvement of 12.93 percentage points over EMT-NAS. Comparison with single-tasking baseline confirms that our transfer rank mechanism is the key driver behind these simultaneous improvements in both search efficiency and final task performance.

Multi-tasking result on MedMNIST: We evaluate KTNAS in a more complex four-task scenario using several MedMNIST subdatasets: PathMNIST, OrganAMNIST, OrganCMNIST, and OrganSMNIST. The detailed comparison results are presented in Table VI. The results show that the EMTO-based methods (EMT-NAS and KTNAS) are more search-efficient than the single-tasking baseline, demonstrating that knowledge transfer can effectively accelerate the search process.

In terms of final model performance, KTNAS consistently outperforms its multi-task counterpart, EMT-NAS, across all four tasks. The most significant improvement is on PathMNIST, where KTNAS achieves a 1.94% higher accuracy (90.54% vs. 88.60%). It also secures consistent gains on OrganAMNIST (94.38% vs. 94.32%), OrganCMNIST (91.57% vs. 91.40%), and OrganSMNIST (80.80% vs. 80.72%). While EMT-NAS

TABLE VI
COMPARISON RESULTS ON MEDMNIST. * DENOTES SELF-IMPLEMENTATION

# Tasks	Model	Search Method	GPU Days (%)↓	Path (%)	Organ_A (%)	Organ_C (%)	Organ_S (%)
1	ResNet-50 [1]	manual	-	86.4	91.6	89.3	74.6
	ResNet-18 [1]	manual	-	84.4	92.1	88.9	76.2
	Auto-sklearn [53]	GO	-	18.6	56.3	67.6	60.1
	AutoKeras [54]	GO	-	86.4	92.9	91.5	80.3
	AutoML	GO	-	81.1	81.8	86.1	70.6
	SI-EvoNAS [29]	EA	-	90.5±0.7	93.0±0.3	91.8±0.4	80.1±0.3
4	Single-Tasking (Baseline) [14]*	EA	+00.0	88.36±1.41	93.64±0.41	91.18±0.48	80.42±0.52
	EMT-NAS [14]*	EMTO	**-9.55**	88.60±3.09	94.32±0.44	91.40±0.73	80.72±0.94
	KTNAS (Ours)	EMTO	-7.93	**90.54±1.31**	**94.38±0.20**	**91.57±0.38**	**80.80±0.63**

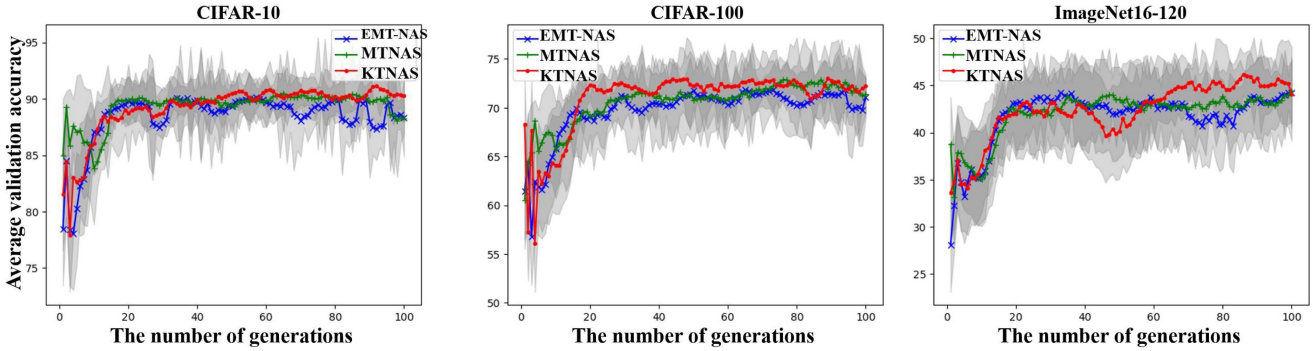

Fig. 5. The average convergence curves on NAS201. The shaded area denotes standard variance over 10 runs.

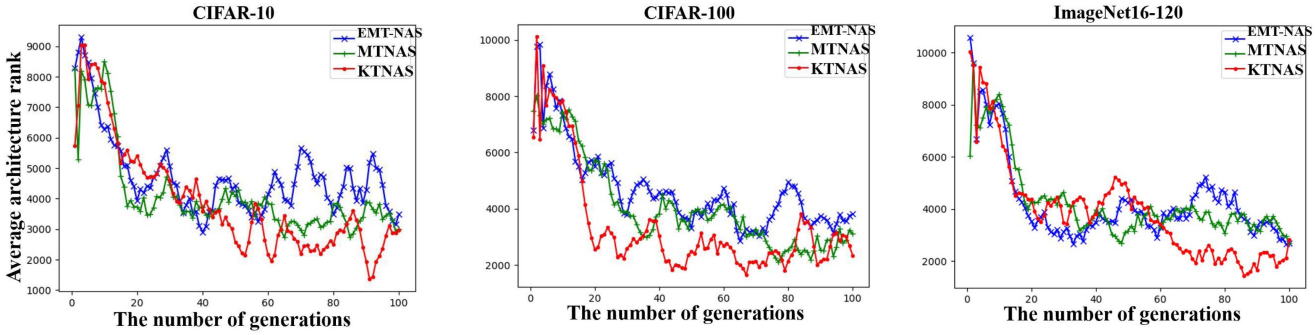

Fig. 6. The average rank of transferred architectures on NAS201.

is slightly more efficient, KTNAS delivers substantially better overall accuracy, highlighting its ability to identify superior-performing architectures.

E. Transfer Performance Analysis

This section provides deeper insights into transfer performance over 3 multi-task NAS. We set terminated generation to 100 and run 10 times with different seeds on NAS201. The validation accuracy and transferred architecture rank on each generation are recorded and the average results of 10 runs are presented.

Fig. 5 displays the average convergence curves for validation accuracy. Across all tasks, the performance of the best-found architectures improves rapidly during the initial 20 generations before stabilizing after approximately the 60th generation. Crucially, KTNAS consistently outperforms both EMT-NAS and MT-NAS, achieving higher validation accuracy not only at the end of the search but also throughout most of the intermediate generations. This highlights the effectiveness of our approach in consistently identifying superior architectures.

Fig. 6 illustrates the average rank of the architectures selected for transfer at each generation. A clear trend is visible across all tasks: the average rank of the transferred architectures steadily decreases as the search progresses, indicating that the algorithms are successfully learning to share higher-quality architectures. Most importantly, KTNAS consistently achieves the best/lowest average rank among the three methods. This result directly validates our core hypothesis: the proposed transfer rank mechanism is a more effective strategy for selecting high-potential architectures for knowledge transfer.

TABLE VII
COMPARISON RESULTS OF ABLATION STUDY ABOUT TWO KEY COMPONENTS OF AQT WHEN PERFORMING KTNAS ON C-10/C-100.

Index	arch2vec	Transfer rank	GPU Days	C-10 Test Acc(%)	C-100 Test Acc(%)
①	✗	✗	**0.46**	96.64±0.18	80.82±0.71
②	✓	✗	0.47	96.63±0.16	80.79±0.10
③	✗	✓	0.47	96.82±0.13	80.85±0.22
④	✓	✓	0.48	**97.12±0.10**	**82.93±0.19**

F. Ablation Study

To isolate the contributions of the key components of our proposed Architecture Transfer via Quantization (ATQ) module, we conduct a comprehensive ablation study on CIFAR-10 and CIFAR-100. The study evaluates four distinct configurations, with results presented in Table VII. The configurations are as follows:

① *Baseline:* KTNAS with no ATQ module, serving as our control group.

② *arch2vec only:* Only the arch2vec component is active. In the ATQ module, architectures are embedded, but the transfer rank mechanism is disabled, meaning no knowledge is actually transferred. This isolates the computational cost of the embedding model.

③ *Transfer rank only:* The transfer rank mechanism is active, but arch2vec is disabled. Architectural similarity is calculated using a naive discrete encoding instead of the learned arch2vec embeddings. This assesses the effectiveness of the ranking mechanism without a sophisticated latent space.

④ *KTNAS with ATQ:* The ATQ module is active with both arch2vec and transfer rank enabled.

From the results, we draw the following conclusions. First, comparing the baseline (①) with the arch2vec-only configuration (②), we observe that incorporating the pre-trained arch2vec model adds negligible search overhead (0.01 GPU Days) but yields no performance improvement on its own.

Second, adding the transfer rank mechanism even with a naive discrete encoding (③) provides a noticeable accuracy boost over the baseline (①) for a minimal increase in search cost. This suggests that the act of structured knowledge transfer is beneficial by enhancing population diversity during the search even when guided by a suboptimal similarity metric.

Finally, the full KTNAS model (④), which combines both arch2vec and transfer rank, achieves the most significant performance gains. It surpasses the baseline (①) by a substantial margin—0.48% on CIFAR-10 and a remarkable 2.11% on CIFAR-100—while incurring only a slight 0.02 GPU Day overhead. Crucially, it also significantly outperforms the transfer-rank-only model (③), demonstrating a powerful synergistic effect. This confirms that while transfer rank is effective, its performance is dramatically improved when combined with arch2vec's smooth, meaningful latent space. This allows for a more accurate calculation of architectural similarity and, consequently, a more effective transfer of high-quality architectural knowledge.

TABLE VIII
PARAMETER SENSITIVITY ANALYSIS ABOUT ELITE RANKING RATIO, THE NUMBER OF SAVED GENERATIONS AND THE NUMBER OF TRANSFERRED INDIVIDUALS ON C-10/C-100.

	Value	C-10		C-100	
		Val Acc(%)	Test Acc(%)	Val Acc(%)	Test Acc(%)
r%	10%	90.84±0.46	97.15±0.04	65.59±0.13	82.88±0.19
	20%	**90.85±0.38**	**97.16±0.05**	**65.60±0.28**	**82.89±0.18**
	30%	90.83±0.33	97.13±0.06	65.57±0.25	82.85±0.14
	40%	90.82±0.41	97.12±0.04	65.54±0.19	82.82±0.12
	50%	90.83±0.20	97.10±0.07	65.25±1.32	82.54±0.06
	60%	90.80±0.62	97.05±0.05	65.20±0.37	82.50±0.09
	70%	90.78±0.55	96.98±0.06	65.15±0.42	82.45±0.11
	80%	90.79±0.71	96.13±0.05	64.84±1.43	81.66±0.15
	90%	90.75±0.68	96.05±0.04	64.80±0.39	81.60±0.13
m	1	90.01±2.55	96.79±0.13	65.47±0.86	82.32±0.21
	2	90.42±1.18	96.95±0.09	65.39±0.92	82.43±0.15
	3	90.83±0.20	97.10±0.07	65.25±1.32	82.54±0.06
	4	90.82±0.74	97.19±0.08	65.39±0.67	82.76±0.12
	5	**90.81±0.86**	**97.27±0.12**	**65.52±0.39**	**83.97±0.19**
	6	90.78±1.02	97.30±0.11	65.60±0.54	84.10±0.21
	7	90.75±1.14	97.33±0.10	65.68±0.62	84.23±0.18
	8	90.72±1.27	97.36±0.09	65.76±0.71	84.36±0.16
	9	90.69±1.35	97.39±0.08	65.84±0.79	84.49±0.14
n	1	89.85±0.95	96.68±0.05	65.10±1.12	82.33±0.12
	2	89.71±1.27	96.71±0.06	65.06±1.42	82.37±0.14
	3	89.77±0.88	96.90±0.05	65.15±1.25	82.45±0.11
	4	**90.83±0.20**	**97.10±0.07**	**65.25±1.32**	**82.54±0.06**
	5	90.70±0.31	97.15±0.06	65.30±0.98	82.60±0.08
	6	90.65±0.42	97.20±0.05	65.35±0.85	82.65±0.07
	7	90.60±0.53	97.25±0.04	65.40±0.72	82.70±0.06
	8	90.27±0.31	96.86±0.03	64.98±0.37	82.06±0.11
	9	90.20±0.28	96.80±0.02	64.90±0.35	82.00±0.09

G. Parameter Sensitivity Analysis

Transfer parameter sensitivity results are given in Table VIII. The best results on both validation accuracy and test accuracy are achieved when elite ranking ratio $r\% = 20\%$, the number of saved generations $m = 5$ and the number of transferred individuals $n = 4$.

We observed that increasing r% from 10% to 20% consistently improved both validation and test accuracy on both datasets. This indicates that a low elite selection is effective for enhancing model performance. However, once r% exceeded 30%, accuracy began to steadily decline. This suggests that an excessively high ratio introduces a large number of lower-quality individuals into the selection pool, which in turn impairs the model's ability to generalize.

The impact of m was highly dependent on dataset complexity. On the more challenging C-100 dataset, increasing m led to a larger number of training samples and a sustained upward trend in test accuracy. In contrast, on the simpler C-10 dataset, the same change caused a slight drop in validation accuracy (from 90.83% to 90.69%). This divergence suggests that while a larger m is beneficial for complex tasks, it may pose an overfitting risk on simpler ones.

n was critical for balancing information transfer. An optimal balance was achieved at n = 4, yielding peak accuracies of 97.10% on C-10 and 82.54% on C-100. When n was below 4, the performance gains were limited, likely due to insufficient information being transferred. Conversely, when n was greater than 7, C-10 validation accuracy dropped significantly to

TABLE IX
COMPARISON ON THE NUMBER OF EVALUATIONS ON NAS201 USING DIFFERENT GRAPH EMBEDDING METHODS.

embeddings	C-10	C-100	ImageNet	Total
node2vec [39]	105.0	26.6	64.4	196.0
CATE [40]	100.5	34.1	59.2	193.8
arch2vec [18]	**100.7**	**27.9**	**60.0**	**188.5**

90.20%, indicating that excessive information transfer can lead to knowledge conflicts or negative interference between models.

Ablation study of embedding methods is shown in IX , including methods tailored for architecture embedding [18], [40]. We ultimately chose arch2vec [18] as the architecture embedding method because of its lowest total cost.

V. CONCLUSION

In this work, we introduced KTNAS, a novel multi-task neural architecture search framework designed to improve cross-task knowledge transfer by explicitly quantifying architecture transferability. The core innovation is our proposed Architecture Transferability Quantifying (ATQ) module. By integrating architecture embeddings method arch2vec with a tailored transfer rank, the ATQ module effectively identifies and leverages promising architectural patterns across tasks. This approach successfully mitigates negative transfer arising from ranking inconsistencies between source and target tasks, a common challenge in transfer learning.

Our extensive experiments on three popular benchmarks (NAS-Bench-201, TransNAS-Bench-101, and DARTs) validate the superiority of KTNAS over state-of-the-art single-task and multi-task NAS methods. KTNAS consistently discovers architectures with higher accuracy while remaining competitive in search efficiency, particularly pronounced in complex tasks. Furthermore, ablation studies confirm that both the architecture embeddings and the transfer rank are critical components contributing to this strong performance.

Our work has several limitations. First, the efficacy of ATQ depends on the quality of the pretrained architecture embeddings, which may not fully capture architectural small differences. Second, the current experiments assume a homogeneous search space across all tasks, limiting direct application to more heterogeneous search spaces. Here are several promising directions for future research: 1) exploring more advanced architecture representation learning methods that jointly model structural and performance; 2) extending the transferability quantification to accommodate heterogeneous search spaces and more diverse task relationships; 3)integrating ATQ with other low-fidelity strategies for enhancing efficiency. Pursuing these avenues will further strengthen the practical applications of transfer-aware MTNAS in real-world.

REFERENCES

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[2] S. Zagoruyko and N. Komodakis, "Wide residual networks," in *Proc. Brit. Mach. Vis. Conf.*, 2016.

[3] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 4700–4708.

[4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetv2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4510–4520.

[5] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2015, pp. 1440–1448.

[6] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Proc. : 18th Int. Conf. Med. Image Comput. Comput.-Assis. Interv.*, Munich, Germany, 2015, pp. 234–241.

[7] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

[8] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8697–8710.

[9] X. Zhou, Z. Wang, L. Feng, S. Liu, K.-C. Wong, and K. C. Tan, "Towards evolutionary multi-task convolutional neural architecture search," *IEEE Trans. Evol. Comput.*, vol. 28, no. 3, pp. 682–695, Jun. 2024.

[10] X. Wan, B. Ru, P. M. Esperança, and Z. Li, "On redundancy and diversity in cell-based neural architecture search," presented at the 10th International Conference on Learning Representations, vol. 25, 2022, Art. no. 29.

[11] Y. Duan et al., "TransNAS-Bench-101: Improving transferability and generalizability of cross-task neural architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 5251–5260.

[12] X. Dong and Y. Yang, "NAS-Bench-201: Extending the scope of reproducible neural architecture search," in *Proc. Int. Conf. Learn. Representations*, 2020.

[13] R. Panda et al., "NASTransfer: Analyzing architecture transferability in large scale neural architecture search," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 10, pp. 9294–9302.

[14] P. Liao, Y. Jin, and W. Du, "Emt-nas: Transferring architectural knowledge between tasks from different datasets," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 3643–3653.

[15] A. Gupta, Y.-S. Ong, and L. Feng, "Multifactorial evolution: Toward evolutionary multitasking," *IEEE Trans. Evol. Comput.*, vol. 20, no. 3, pp. 343–357, Jun. 2016.

[16] Y. Benyahia et al., "Overcoming multi-model forgetting," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 594–603.

[17] H. Chen, H.-L. Liu, F. Gu, and K. C. Tan, "A multiobjective multitask optimization algorithm using transfer rank," *IEEE Trans. Evol. Comput.*, vol. 27, no. 2, pp. 237–250, Apr. 2023.

[18] S. Yan, Y. Zheng, W. Ao, X. Zeng, and M. Zhang, "Does unsupervised architecture representation learning help neural architecture search?," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 12486–12498.

[19] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," in *Proc. Int. Conf. Learn. Representations*, 2018.

[20] H. Pham, M. Guan, B. Zoph, Q. Le, and J. Dean, "Efficient neural architecture search via parameters sharing," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4095–4104.

[21] S. Xie, H. Zheng, C. Liu, and L. Lin, "SNAS: Stochastic neural architecture search," in *Proc. Int. Conf. Learn. Representations*, 2018.

[22] E. Real et al., "Large-scale evolution of image classifiers," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 2902–2911.

[23] H. Liu, K. Simonyan, O. Vinyals, C. Fernando, and K. Kavukcuoglu, "Hierarchical representations for efficient architecture search," in *Proc. Int. Conf. Learn. Representations*, 2018.

[24] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, no. 01, pp. 4780–4789.

[25] Y. Sun, B. Xue, M. Zhang, and G. G. Yen, "Completely automated CNN architecture design based on blocks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 4, pp. 1242–1254, Apr. 2020.

[26] J.-Y. Li, Z.-H. Zhan, J. Xu, S. Kwong, and J. Zhang, "Surrogate-assisted hybrid-model estimation of distribution algorithm for mixed-variable hyperparameters optimization in convolutional neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 5, pp. 2338–2352, May 2023.

[27] Z. Lu et al., "Nsga-NET: Neural architecture search using multi-objective genetic algorithm," in *Proc. Genet. Evol. Comput. Conf.*, 2019, pp. 419–427.

[28] Z. Yang et al., "Cars: Continuous evolution for efficient neural architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 1829–1838.

[29] H. Zhang, Y. Jin, R. Cheng, and K. Hao, "Efficient evolutionary search of attention convolutional networks via sampled training and node inheritance," *IEEE Trans. Evol. Comput.*, vol. 25, no. 2, pp. 371–385, Apr. 2021.
[30] R. Pasunuru and M. Bansal, "Continual and multi-task architecture search," in *Proc, 57th Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 1911–1922.
[31] R. Cai and J. Luo, "Multi-task learning for multi-objective evolutionary neural architecture search," in *Proc. 2021 IEEE Congr. Evol. Comput.*, 2021, pp. 1680–1687.
[32] X. Zhou, S. Liu, A. Qin, and K. C. Tan, "Evolutionary neural architecture search for transferable networks," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 9, no. 2, pp. 1556–1568, Apr. 2025.
[33] N. Li, B. Xue, L. Ma, and M. Zhang, "Automatic fuzzy architecture design for defect detection via classifier-assisted multiobjective optimization approach," *IEEE Trans. Evol. Comput.*, early access, Jan., 16, 2025, doi: 10.1109/TEVC.2025.3530416.
[34] W. Li, Y. Chen, Y. Dong, and Y. Huang, "A solution potential-based adaptation reference vector evolutionary algorithm for many-objective optimization," *Swarm Evol. Comput.*, vol. 84, 2024, Art. no. 101451.
[35] X. Xue, G. Mao, S. Kumari, and Z. Liu, "Multi-objective genetic programming assisted stochastic deep reinforcement learning for dynamic knowledge integration in transportation networks," *IEEE Trans. Intell. Transp. Syst.*, early access, Feb., 28, 2025, doi: 10.1109/TITS.2025.3545572.
[36] A. Gupta, Y.-S. Ong, L. Feng, and K. C. Tan, "Multiobjective multifactorial optimization in evolutionary multitasking," *IEEE Trans. Cybern.*, vol. 47, no. 7, pp. 1652–1665, Jul. 2017.
[37] L. Feng et al., "Evolutionary multitasking via explicit autoencoding," *IEEE Trans. Cybern.*, vol. 49, no. 9, pp. 3457–3470, Sep. 2019.
[38] J. Lin, H.-L. Liu, B. Xue, M. Zhang, and F. Gu, "Multiobjective multitasking optimization based on incremental learning," *IEEE Trans. Evol. Comput.*, vol. 24, no. 5, pp. 824–838, Oct. 2020.
[39] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 855–864.
[40] S. Yan, K. Song, F. Liu, and M. Zhang, "CATE: Computation-aware neural architecture encoding with transformers," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 11670–11681.
[41] W. Wen, H. Liu, Y. Chen, H. Li, G. Bender, and P.-J. Kindermans, "Neural predictor for neural architecture search," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2020, pp. 660–676.
[42] M. S. Abdelfattah, A. Mehrotra, Ł. Dudziak, and N. D. Lane, "Zero-cost proxies for lightweight NAS," in *Proc. Int. Conf. Learn. Representations*, 2021.
[43] L. Wang, S. Xie, T. Li, R. Fonseca, and Y. Tian, "Sample-efficient neural architecture search by learning actions for monte carlo tree search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 9, pp. 5503–5515, Sep. 2022.
[44] J. Wu et al., "Stronger NAS with weaker predictors," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 28904–28918.
[45] Z. Lu, K. Deb, E. Goodman, W. Banzhaf, and V. N. Boddeti, "Nsganetv2: Evolutionary multi-objective surrogate-assisted neural architecture search," in *Proc. 16th Eur. Conf. Comput. Vis.*, Glasgow, U.K., 2020, pp. 35–51.
[46] C. Liu et al., "Progressive neural architecture search," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 19–34.
[47] C. Peng, Y. Li, R. Shang, and L. Jiao, "RecNAS: Resource-constrained neural architecture search based on differentiable annealing and dynamic pruning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 2, pp. 2805–2819, Feb. 2024.
[48] Y. Xue, B. Hu, and F. Neri, "A surrogate model with multiple comparisons and semi-online learning for evolutionary neural architecture search," *IEEE Trans. Emerg. Topics Comput. Intell.*, early access, Mar., 20, 2025, doi: 10.1109/TETCI.2025.3547621.
[49] O.-C. Granmo, S. Glimsdal, L. Jiao, M. Goodwin, C. W. Omlin, and G. T. Berge, "The convolutional Tsetlin machine," 2019, *arXiv:1905.09688*.
[50] L. Taylor, A. King, and N. Harper, "Robust and accelerated single-spike spiking neural network training with applicability to challenging temporal tasks," 2022, *arXiv:2205.15286*.
[51] Y. Sun, L. Zhang, and H. Schaeffer, "Neupde: Neural network based ordinary and partial differential equations for modeling time-dependent data," in *Proc. Math. Sci. Mach. Learn.*, 2020, pp. 352–372.
[52] P. Jeevan and A. Sethi, "Wavemix: Resource-efficient token mixing for images," 2022, *arXiv:2203.03689*.
[53] M. Feurer, A. Klein, K. Eggensperger, J. Springenberg, M. Blum, and F. Hutter, "Auto-sklearn: Efficient and robust automated machine learning," *Autom. Mach. Learn.*, 2019, Art. no. 113.
[54] H. Jin, Q. Song, and X. Hu, "Auto-Keras: An efficient neural architecture search system," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 1946–1956.

Tingjie Zhang received the B.S. degree from the School of Automation, Guangdong University of Technology, Guangzhou, China, in 2022, and the M.S. degree from School of Mathematics and Statistics, Guangdong University of Technology, in 2025. His current research interests include evolutionary optimization and neural architecture search.

Hai-Lin Liu (Senior Member, IEEE) received the B.S. degree in mathematics from Henan Normal University, Xinxiang, China, in 1984, the M.S. degree in applied mathematics from Xidian University, Xi'an, China, in 1989, and the Ph.D. degree in control theory and engineering from the South China University of Technology, Guangzhou, China, in 2002.He was a Postdoctoral Fellow with the Institute of Electronic and Information, South China University of Technology. He is currently a Full Professor with the School of Mathematics and Statistics, Guangdong University of Technology, Guangzhou, China. He has authored or coauthored more than 100 research papers in journals and conferences. His research interests include evolutionary computation and optimization, wireless network planning and optimization, and their applications. He is also an Associate Editor for IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTATIONAL INTELLIGENCE. He is the Editorial Board Member of *Applied Soft Computing* and *Memetic Computing*.

Songyi Xiao received the B.S. and M.S. degrees from the School of Information Engineering, Nanchang Institute of Technology, Nanchang, China, in 2018 and 2021, respectively. He is currently working toward the Ph.D. degree in control science and engineering with the School of Automation, Guangdong University of Tech nology, Guangzhou, China. His research interests include evolutionary optimization, multi-objective optimization, and neural architecture search.

Lei Chen received the B.S. degree in mathematics and applied mathematics from Henan University, Kaifeng, China, and the Ph.D. degree in control science and engineering from the Guangdong University of Technology, Guangzhou, China, in 2011 and 2019, respectively. He is currently an Associate Professor with the School of Mathematics and Statistics, Guangdong University of Technology. His research interests include evolutionary multi-objective optimization, bi-level optimization, and its applications.